

Agentic System Architecture for Zendesk's Autonomous Customer Service Agents

Problem Analysis and System Requirements

Zendesk already deploys autonomous agents that plan multi-step solutions, call live APIs to refund orders or modify bookings, and autonomously close tickets. The core agent uses an **iterative planner** that decomposes the high-level goal into ordered actions, pulls skills from a registry, executes them, evaluates the outcome, and refines the plan. Successful patterns are synthesised into new skills.

However, several open issues remain:

- **Ambiguous goal decomposition:** The current planner works when user intent is clear, but tickets often contain vague or conflicting requests. Without clarifying questions, the plan can diverge or oscillate.
- **Memory interference across concurrent sessions:** Multi-agent workflows require shared context. Analysis shows that **36.9 % of multi-agent failures are caused by inter-agent misalignment** — agents duplicating or contradicting one another because they operate on different versions of reality [Mem0 multi-agent memory](#). When each agent maintains its own context window, they may duplicate API calls or deliver inconsistent answers [Mem0 multi-agent memory](#).
- **Over-eager skill acquisition:** The self-learning loop automatically synthesises new skills from successful traces, but without selectivity this registry becomes noisy and skills overlap or conflict.
- **Multi-agent delegation (A2A):** Tasks such as billing, returns and escalation require one agent to hand off sub-tasks to another specialist. There is no design yet for structured delegation, role boundaries or hand-back protocols.
- **Domain-specialized models:** Zendesk plans to train its own customer-service LLMs using **reinforcement learning** on production trajectories. Offline RL methods (such as OREO) show that conventional preference-based fine-tuning (DPO) struggles with sparse rewards and credit assignment; OREO jointly learns a policy and value function using the soft Bellman equation, avoiding the need for paired preference data and outperforming baselines on multi-step reasoning tasks [OREO paper](#). The system must collect and process production traces, define reward signals and run scalable offline/online RL pipelines.
- **Evaluation and regression detection:** Agent evaluation must inspect reasoning chains, tool selection and execution paths—not just final outputs. Braintrust's framework emphasises layered metrics: plan quality, plan adherence, tool selection accuracy, tool correctness, path validity, task completion, efficiency and safety [Braintrust agent evaluation framework](#). Continuous evaluation harnesses with regression gates must be integrated into CI.
- **Enterprise-grade guardrails:** Autonomous agents expose unique security risks: prompt injection, tool misuse, cascading hallucinations and reward hacking. Enkrypt AI's taxonomy distinguishes **input and output guardrails** that pre-filter requests and responses [Enkrypt AI agent guardrails](#), and a **guardrails-by-design approach** that embeds secure architecture, least-privilege permissions, security-first prompting and integrated monitoring [Enkrypt AI agent guardrails](#). These controls must operate across thousands of concurrent sessions without incurring significant latency.

Design Trade-offs

Memory architecture: Three patterns exist for multi-agent memory [Mem0 multi-agent memory](#):

1. *Centralised memory* – a single shared store simplifies consistency but becomes a bottleneck and single point of failure as the number of agents increases.
2. *Distributed memory* – each agent maintains its own store and selectively syncs updates; this scales but incurs consistency challenges.
3. *Hybrid memory* – the practical compromise used in most systems; frequently accessed context is centralised (e.g., ticket state), while specialised context lives in per-agent stores with versioning.

The system must also respect memory scopes—user, session, agent and application—and design them early, because changing scoping later is costly [Mem0 multi-agent memory](#). A concurrency control layer must prevent stale writes and coordinate updates.

Offline vs online RL: Online RL methods like PPO require on-the-fly data collection and are expensive; offline methods such as **OREO** leverage logged trajectories, learn a policy and value function via the soft Bellman equation, and require no paired preference data [OREO paper](#). However, pure offline methods may overfit to logged behaviour and fail to explore; a hybrid approach with occasional online rollouts is planned.

Skill acquisition: Adding every successful plan as a new skill leads to bloat. Skills should undergo evaluation using layered metrics and only graduate to the registry when they consistently improve resolution metrics. A retention policy should prune unused or redundant skills.

A2A delegation: Hand-offs must be explicit. Each agent advertises capabilities, accepts sub-tasks with clear contracts (inputs, expected outputs and error codes) and returns control to the delegating agent upon completion. Delegation should involve shared memory segments rather than full conversation histories to avoid “context dumping” and token bloat [Mem0 multi-agent memory](#).

Proposed Agentic System Architecture

1. Planning and Control Layer

At the top sits a **meta-planner** (Figure 1) that receives the user ticket and constructs a hierarchical plan. It uses goal-recognition heuristics, domain ontologies and, when the goal is ambiguous, issues clarification prompts. The planner iteratively decomposes tasks into actions, referencing a **skill registry** for reusable functions (e.g., “lookup order”, “process refund”). Plans are saved to shared memory and annotated with success criteria (objectives and guard conditions).

2. Skill Registry and Execution Engine

Skills are modular functions with metadata (preconditions, input schema, output schema, side-effects). Skills can be handcrafted or synthesised from previous runs; however, new skills must pass evaluation criteria (plan quality, tool correctness etc.) before being added. An **execution engine** interprets the plan, selects skills, invokes external APIs, handles retries and monitors outcomes. If a skill fails, the engine records the trace and triggers the planner to refine the plan.

3. Multi-Tier Memory

The architecture adopts a **hybrid memory** pattern:

- **Working memory** holds the active conversation, current plan and intermediate tool responses. It resembles the **CoALA framework** - working memory, long-term memory and semantic/procedural knowledge.
- **Long-term agent memory** stores episodic knowledge (past tickets), semantic facts and procedural knowledge (skills). This tier is persistent and cross-session.
- **Shared ticket memory** acts as the central store for the ticket state, accessible by all agents involved in the session. It ensures that agents operate on the same reality and reduces duplication and inconsistent responses [Mem0 multi-agent memory](#). Access control lists and versioning prevent race conditions.
- **Per-agent memory** stores private context, drafts and intermediate reasoning. Agents synchronise with the shared memory through transactions. Changes require commit operations that enforce validation rules.

4. Multi-Agent Coordination (A2A)

Zendesk's agents specialise (e.g., **BillingAgent**, **PolicyAgent**, **EscalationAgent**). The **A2A protocol** defines:

1. **Capability advertisement** – agents register their capabilities and required inputs.
2. **Task delegation** – the planner or an agent uses the capability registry to delegate sub-tasks. Delegation includes a pointer to the shared memory segment for the ticket and a contract describing expected outcomes and fallback paths.
3. **Result hand-back** – the sub-agent writes results to shared memory and signals completion or failure.
4. **Timeouts and escalation** – if the sub-task fails, times out or deviates from policy, the supervisor agent can intervene.

This pattern allows specialised agents to handle complex tasks without overwhelming a single model and prevents duplication of work.

5. RL-Based Model Training Pipeline

Zendesk's goal is to train a **domain-specialised LLM** for customer service. Key steps:

1. **Data collection** – instrument the production system to log full trajectories: user messages, plans, tool calls, API responses, outcomes (resolved vs escalated), user satisfaction, resolution time and any escalations. Logs must exclude sensitive data and include reward signals (e.g., success, CSAT, cost).
2. **Reward modeling and shaping** – derive rewards from resolution success, escalation avoidance, user satisfaction and efficiency. Reward shaping mitigates sparse rewards and encourages step-wise credit assignment.
3. **Offline RL training** – use algorithms like **OREO** which jointly learn a policy and value function using the soft Bellman equation and can work from unpaired trajectories [OREO paper](#). OREO avoids the need for pairwise preference data and provides better credit assignment in multi-step reasoning

[OREO paper](#).

4. **Hybrid training** – supplement offline RL with occasional online rollouts in simulation environments to explore new strategies and update the reward model.
5. **Evaluation and selection** – evaluate candidate models using the agent evaluation harness (see below) to ensure improvements generalise beyond offline logs.

6. Evaluation Harness and Continuous Integration

Evaluation is not a one-time exercise; it must run continuously. The harness implements Braintrust's layered metrics:

- **Reasoning metrics:** plan quality, plan adherence, tool selection accuracy and deviation from plan [Braintrust agent evaluation framework](#).
- **Action metrics:** tool correctness, argument correctness and execution path validity [Braintrust agent evaluation framework](#).
- **End-to-end metrics:** task completion, step efficiency, latency and cost [Braintrust agent evaluation framework](#).
- **Safety metrics:** prompt injection resilience, policy adherence and bias detection [Braintrust agent evaluation framework](#).

The harness uses **tracing** to capture every decision, tool call and response [Braintrust agent evaluation framework](#). Each commit triggers the harness on a suite of scenario-based tests drawn from real tickets; regression gates block deploys when metrics drop.

7. Guardrails and Supervisory Layer

Security and safety controls are layered:

1. **Black-box guardrails** wrap around the agent and implement input and output filtering. Input guardrails block harmful prompts, detect PII, filter sensitive topics and prevent prompt injection [Enkrypt AI agent guardrails](#). Output guardrails filter responses for harmful, biased or factually incorrect content [Enkrypt AI agent guardrails](#).
2. **Guardrails by design** embed security into the architecture. Secure workflows incorporate threat modelling and proper authentication and authorisation [Enkrypt AI agent guardrails](#). The **principle of least privilege** grants agents only the permissions they need [Enkrypt AI agent guardrails](#). **Security-first prompting** and **integrated monitoring** with audit trails and real-time threat detection provide ongoing visibility [Enkrypt AI agent guardrails](#).
3. **Governance guardrails** ensure agents optimise for intended outcomes rather than proxy metrics. Input guardrails detect prompt injection that could change the agent's goal, and tool guardrails require human approval for goal or policy changes [Enkrypt AI agent guardrails](#). Version control for prompts and models plus behavioural drift monitoring mitigate policy drift [Enkrypt AI agent guardrails](#).

8. Observability and Logging

Comprehensive logging tracks every plan, tool call, API response and internal error. Logs feed the evaluation harness, RL training pipelines and guardrail monitoring. A dashboard surfaces metrics across reasoning and action layers as well as cost and latency. Observability also enables anomaly detection (e.g., sudden spike in API calls) and triggers automated responses.

Critical Design Decisions

- **Hybrid memory and concurrency control** – ensures consistency across agents and avoids token bloat while scaling to thousands of sessions. Scoping dimensions (user, session, agent, application) ensure agents see only relevant information [Mem0 multi-agent memory](#).
- **Selective skill acquisition** – gating new skills through evaluation harness prevents registry bloat and maintains quality. A retention policy periodically prunes unused or overlapping skills.
- **Hierarchical planning with clarification** – ambiguous goals trigger clarifying questions before decomposition. Planning uses introspection to verify that sub-tasks align with the top-level objective; if not, it revises the plan.
- **A2A delegation protocol** – explicit contracts and shared memory pointers standardise how tasks are handed off, avoid context dumping and enable specialised agents to collaborate.
- **RL training pipeline** – offline RL (OREO) is chosen to leverage logged data and reduce the cost of online interaction [OREO paper](#). Reward shaping uses real business metrics to align agent behaviour with customer satisfaction while avoiding reward hacking.
- **Continuous evaluation and guardrails** – evaluation harness integrated into CI ensures regressions are caught early. Layered guardrails provide both fast filtering (black-box) and deep safety controls (by design) [Enkrypt AI agent guardrails](#), [Braintrust agent evaluation framework](#).

Figure 1. System Overview

The system comprises a meta-planner, skill registry and execution engine interacting with a hybrid memory layer. Specialized agents coordinate through the A2A protocol and share context via shared ticket memory. A supervisory layer implements guardrails and monitors for security and policy compliance. An evaluation harness continuously tests reasoning and action metrics, while the RL training pipeline ingests execution traces, trains domain-specialized models and feeds improvements back into the system.